

Freshness-aware edge replay for MQTT digital-twin synchronization

Final research-phase reproduction and methodology report

Prepared 7 October 2026 • Cloud Computing research project • Muhammad Sameer and Humayun Bilal (proposal team names; authorship/contributions require human confirmation).

Status: executed local research package and preliminary manuscript material, including an additional Linux Docker attack reproduction and real packet-impairment sweep. Exact original-paper numerical replication remains unavailable; executed AWS validation is reported in Section 13. Mentor approval is reported by the team on 7 October 2026. This PDF is not the final instructor-template Overleaf manuscript.

Abstract

Buffered telemetry can preserve a replay opportunity while leaving a digital twin's current state stale. We audit and execute the feasible MQTT core of a recent communication-monitoring study, then evaluate fair latest-state coalescing with a sequence-checked receiver and separate durable local history. The empirical package contains 108 actual MQTT loopback runs across three QoS levels and four application-shaped uplink conditions, plus 1,200 application-simulation executions across load, scale and policy variations. In the live outage condition, mean Age of Information decreases from 1.852 s with FIFO to 0.503 s with the proposed policy (72.8%); finite-horizon delivery remains approximately 53.9%. A fair epoch-only baseline reaches similar freshness in short live tests, so universal superiority is not claimed. Every generated live sample is preserved locally; intermediate updates are not all delivered remotely. Removing the receiver guard causes 5,945 regressions across the unguarded simulation runs. The results support distinguishing current-state freshness from traffic rate and history completeness. Supplemental Linux packet-impairment and AWS Internet experiments are reported below. Production durability and attack-classification accuracy remain outside the measured scope.

Reproducibility: [Project repository](#); [research changes and evidence](#).

1. Problem, motivation and contribution

A twin receiving old updates can appear active while its state is obsolete. For a current-state service, replay order and per-device coverage matter in addition to communication reliability. We investigate this application-layer trade-off within the existing Digital Twin + cloud/MQTT project. The cloud component is an independently deployable MQTT subscriber/current-state service; the executed benchmark co-locates it with the edge and broker. The initial benchmark co-locates services; Section 13 adds actual cloud transport. Physical measurements are outside this study.

Our contribution is a reproducible systems experiment combining committed local audit history, per-device latest-state coalescing, round-robin dispatch and a monotonic receiver. These elements are established techniques. We claim a scoped implementation/evaluation contribution and a paper/code reproducibility audit; no theoretical novelty or globally optimal scheduling result is asserted.

2. Reference selection and initial reproduction

Reference authors: Cláudio Rodrigues, Waldir S. S. Júnior, Wilson Oliveira and Isomar Lima. Venue: *Sensors* 25(24), 7476; year: 2025. [Paper and publisher record](#); [official GitHub artifact](#). The separate initial reproduction report supplies the full data/environment inventory, transcribed published Table 2 statistics, installation problems and item-by-item deviations.

The selected reference is Rodrigues et al., *A Data Rate Monitoring Approach for Cyberattack Detection in Digital Twin Communication*, *Sensors* 25(24), 7476 (2025), DOI 10.3390/s25247476. The official artifact is pinned to commit 43346f60a9f41392611242ff3cb7589d7a758aa7. The full selection rationale, original environment/settings and rejected alternatives are in the initial audit. That audit is part of this report package and avoids treating paper descriptions as executed

results.

The native adapter runs original MQTT sensor, receiver and monitor function bodies, retains original calibration timing, and replaces Docker addressing and the broker. A benign telemetry burst exercises the communication-rate monitor. It is not a reproduction of the reference's attacks or packet-level numerical figures.

Measured native evidence: 283 observed MQTT events, 200 deliberately generated benign-burst messages, and 92.82 s elapsed. The observed burst is 16.37 application messages/s. The independent paper-rule implementation, calibrated on this run's observed sensor-message bins, yields 3.235 messages/s; the observer maximum is 4. The original monitor's own device-to-twin calibration maximum is 3. Observer and monitor windows/capture boundaries differ, so their thresholds need not match.

The first native attempt failed to activate calibration because its completion message did not arrive. The repaired adapter resends only that missing control message using a running network loop and QoS1; this was required in the successful run as well. UTF-8 logging is another native adaptation. Failed logs, successful logs, source hashes and message events remain inspectable. Code inspection also found the maximum-threshold rule, unpinned dependencies, Paho API/version disagreement and the difference between application counters and wire PPS/BPS. None of the published Table 2 values is presented as a measurement of this run.

3. Literature-grounded gap and research question

The 16-paper literature matrix covers platform integration, cloud processing, network-aware twins, operational resilience, synchronization budgets, AoI/AoDT, scheduling and reproducible benchmarks. It explicitly includes prior freshness optimization. A statement that digital-twin research has ignored freshness would be indefensible. The selected rate-oriented artifact, however, lacks generation-age-aware current-state application and a fair recovery outbox. Whether its traffic-rate signals reflect useful current-state recovery is therefore untested in that implementation.

RQ: Under constrained edge dispatch and temporary disconnection, how does fair latest-state coalescing with monotonic application compare with FIFO replay and expiry in state age, error, delivery completeness and resource cost?

Objectives are to reproduce the feasible core with an honest audit; implement and preserve matched baseline/proposed policies; quantify freshness/completeness/recovery/resources; and test expiry and receiver-guard ablations. The complete thematic synthesis, review-depth labels and citations are in docs/LITERATURE_REVIEW.md.

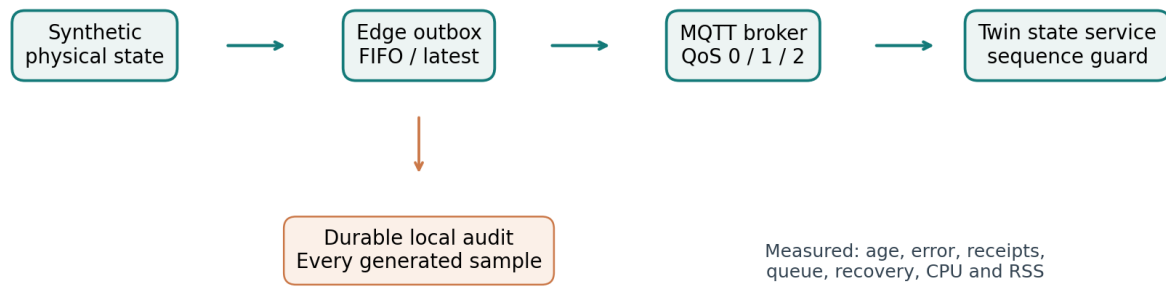
4. Methodology

Messages contain device ID, increasing sequence number, nominal generation time and an analytically known state value. Current state is $\sin(0.3 \cdot t + \text{device})$. No learned model or physical process prediction is used. The receiver's held value and generation time are compared with current analytical ground truth.

AoI is time since the latest applied update was generated. Mean and p95 AoI use device-time observations after warm-up. Stale fraction uses a one-second threshold chosen for this experiment. MAE is unitless held-state error. Delivery ratio divides unique received updates by all generated updates, including intentionally suppressed ones. Synchronization latency includes edge waiting. Recovery requires every device's age to return to at most one second; unrecovered runs are censored rather than silently omitted.

FIFO preserves queued updates in order. Epoch-only staging drops old eligible work and rotates live sensor emission priority. Expiry drops queued updates older than one second in the simulation. The proposal persists all samples in a local audit table and retains only each device's newest queued live update. Round-robin service avoids permanent device priority. A sequence guard rejects duplicate/older application updates. The guard ablation changes only receiver acceptance.

SQLite uses WAL/FULL synchronization and non-reused outbox IDs. Publication-complete acknowledgement precedes deletion, preventing an old acknowledgement from deleting a new replacement. MQTT publisher completion is not a cloud-state commit acknowledgement. The local audit is not delivered to a cloud archive. Commands/events requiring lossless history are outside latest-value coalescing semantics.



5. Experimental design and environment

The live design contains 108 runs: three seeds $\times$ QoS 0/1/2 $\times$ healthy/degraded/overloaded/outage $\times$ epoch-only/FIFO/latest. Each runs six measured seconds after subscriber setup; one second is excluded from freshness metrics. Four devices generate 20 Hz each. Nominal dispatch caps are 160, 60, 30 and 120 messages/s; configured application delay/jitter and a no-dispatch interval from 1.5 to 3.5 seconds complete the scenarios. Already-published messages may drain after the finite horizon; unsent outbox backlog does not. Nominal service caps do not guarantee achieved throughput because disk commits, acknowledgement waits and Windows scheduling consume time.

The simulation contains 1,200 runs: four/sixteen devices $\times$ 2/5/10 Hz $\times$ four conditions $\times$ ten seeds $\times$ five policies. Duration is 60 virtual seconds with five seconds excluded. Its synthetic delay/jitter is keyed by message ID and seed for matched randomness. This model does not implement TCP or MQTT handshakes and is never used to infer simulated QoS packet-loss behavior. Seed-zero raw events are retained for every condition, with per-run records for all repetitions.

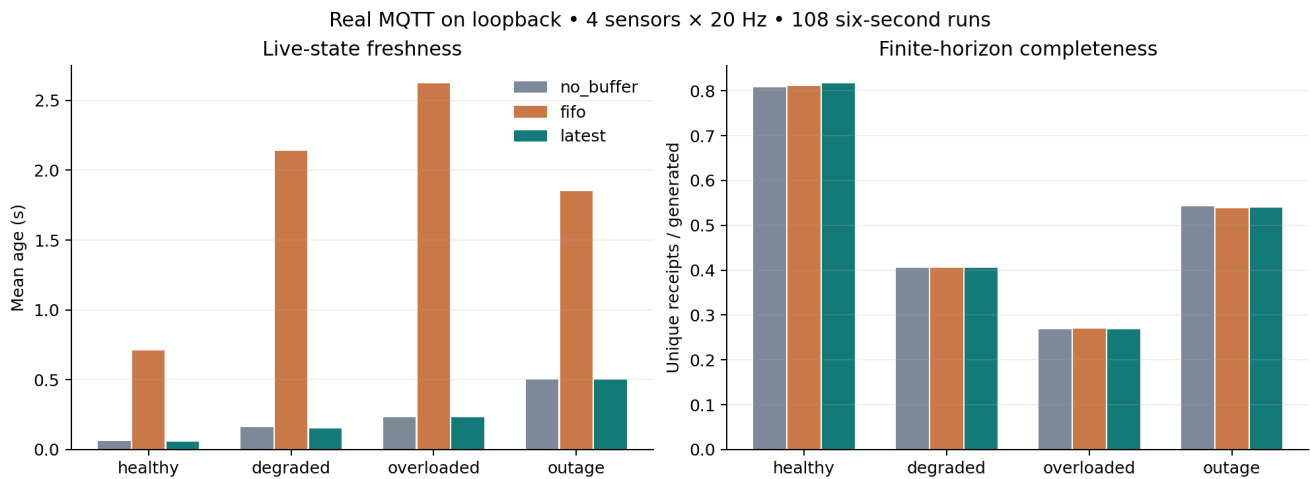
Executed host: Windows build 26200, AMD64 family 23/model 8, six physical/twelve logical CPUs, approximately 15.93 GiB RAM; Python 3.10.9, Paho 2.1.0, AMQTT 0.11.3. The package/environment snapshots contain all versions. Broker/subscriber/edge use loopback. During the early live sweep, background native-core/simulation jobs ran on the shared desktop; randomized order reduces systematic policy grouping but does not eliminate host-load confounding. Costs are descriptive measurements, not isolated broker benchmark claims.

6. Actual MQTT results

The following table averages QoS within each seed before summarizing seeds. Full QoS-specific tables and paired differences are included in the processed results.

Scenario	Policy	Mean age (s)	p95 age (s)	Delivery (%)	MAE
degraded	fifo	2.138	3.469	40.49	0.408
degraded	latest	0.152	0.229	40.56	0.028
degraded	no_buffer	0.162	0.254	40.53	0.030
healthy	fifo	0.709	1.139	81.04	0.128
healthy	latest	0.056	0.094	81.62	0.010
healthy	no_buffer	0.062	0.124	80.72	0.011
outage	fifo	1.852	2.770	53.87	0.349

Scenario	Policy	Mean age (s)	p95 age (s)	Delivery (%)	MAE
outage	latest	0.503	1.868	53.91	0.098
outage	no_buffer	0.502	1.868	54.21	0.096
overloaded	fifo	2.621	4.250	26.94	0.502
overloaded	latest	0.232	0.350	26.88	0.043
overloaded	no_buffer	0.232	0.356	26.81	0.043



The proposed policy reduces outage mean age by 72.8% against FIFO. It restores the all-device one-second age criterion in about 0.120 s after reconnection; all nine FIFO outage runs remain unrecovered at the measurement horizon. This is recovery of held current state, not backlog clearance or complete remote history.

The fair epoch-only baseline matches or slightly exceeds latest-state freshness in the outage/overloaded conditions. Its lack of historical replay is acceptable for this short current-state workload but cannot satisfy a cloud history-completeness requirement. The proposal adds local history and persistence of queued latest state; the current experiment does not demonstrate improved remote archive completeness over that baseline. Differences between QoS levels on this local path do not establish WAN reliability advantages.

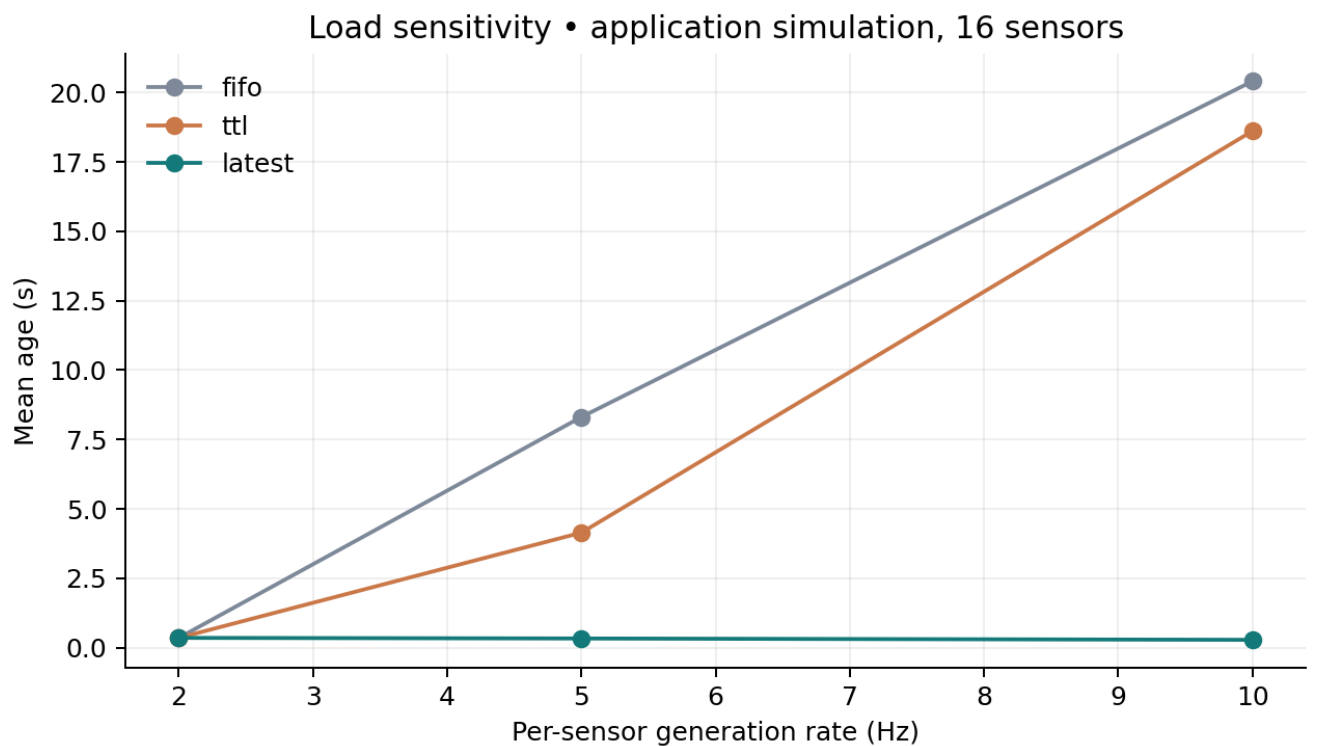
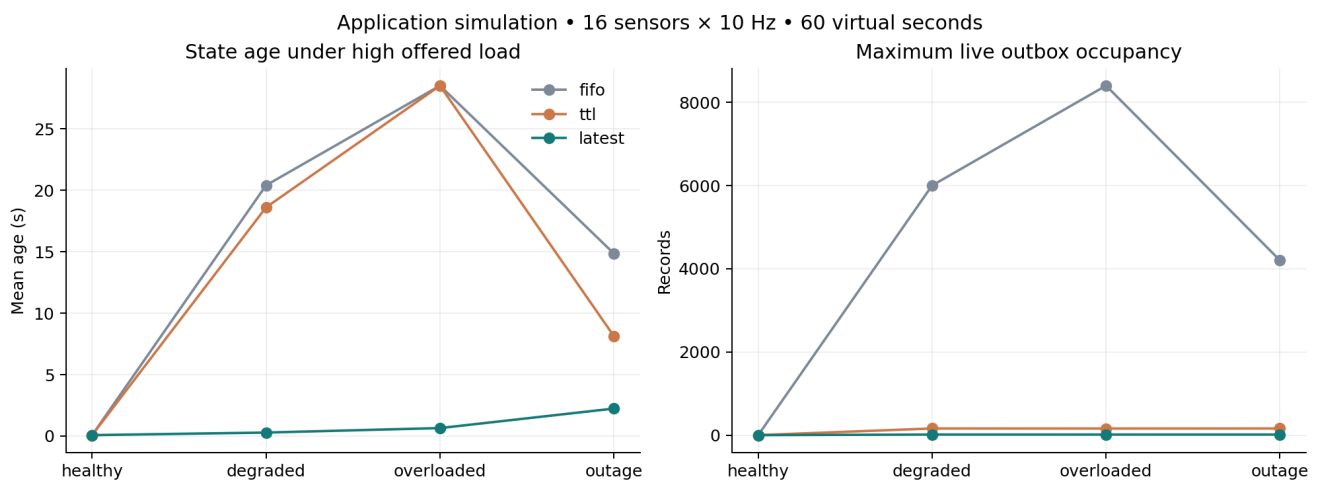
Even the nominal healthy condition does not transmit every generated update within the finite horizon. The actual native service rate is limited by desktop scheduling/storage and implementation overhead. The FIFO age gap in healthy runs therefore reflects a measured end-to-end application bottleneck, not intrinsic healthy-network MQTT latency.

7. Simulation sensitivity and ablation

At the largest offered load (16 devices × 10 Hz), the complete policy comparison is:

Scenario	Policy	Mean age (s)	Delivery (%)	Max queue
degraded	fifo	20.405	37.43	6002
degraded	latest	0.283	37.44	16
degraded	ttl	18.611	37.43	162
healthy	fifo	0.074	99.95	2
healthy	latest	0.074	99.95	2

Scenario	Policy	Mean age (s)	Delivery (%)	Max queue
healthy	ttl	0.074	99.95	2
outage	fifo	14.855	56.18	4202
outage	latest	2.236	56.18	16
outage	ttl	8.134	56.18	162
overloaded	fifo	28.518	12.45	8401
overloaded	latest	0.648	12.45	16
overloaded	ttl	28.514	12.45	161



Higher load produces large FIFO age/backlog even when its receipt ratio resembles the proposal's. Expiry reduces queue size but does not reliably preserve per-device freshness in these periodic workloads. Source/service timing aliasing can select a persistent subset of devices, particularly for simulation epoch-only and expiry controls; conclusions against FIFO are stronger

than comparisons to those secondary controls.

Removing the sequence guard produces 5,945 regressions across the 240 unguarded runs. The guarded proposal has none. Some conditions show zero guard benefit because they do not reorder same-device updates; all conditions, including these null cases, are retained. The count is an application-model outcome, not an MQTT protocol-violation count. Only current-state queue entries are bounded by device count; local audit storage grows with generated history.

8. Runtime, resource and communication costs

Policy	CPU seconds/run	Runner RSS (MiB)	Broker RSS (MiB)	Payload bytes/run
fifo	0.576	28.98	4.73	18541
latest	0.579	29.07	4.73	18871
no_buffer	0.551	29.02	4.72	18807

Values average all final live runs for each policy. CPU seconds include runner threads/SQLite/sampling, but exclude broker CPU and setup/drain. RSS values are means of sampled per-run peaks. Payload bytes exclude protocol headers/acknowledgements and cannot be called wire bandwidth. The simulation consumes 277.02 measured process CPU seconds and 291.28 wall seconds across its 1,200 runs; each run advances 60 virtual seconds.

The proposal's live queue maximum is four rows; FIFO reaches 353 rows. Comparable average runner RSS/CPU are measured on this host, not evidence of negligible overhead on embedded hardware. Every method uses the common audit path, so the epoch-only baseline is not a minimal publisher implementation. No watts, joules, cloud spend or disk-lifetime claim is made.

9. Verification, integrity and evidence

Raw-event and compressed-database checks independently verify 51,840 generated updates, 26,247 published updates and 26,247 received updates. All 108 database histories contain the expected unique device/sequence pairs. Independent event-integral mean age differs from sampled means by at most 0.009665 s. Six correctness tests pass for restart persistence, FIFO/TTL semantics, replacement IDs, late-ack safety, fair dispatch, receiver monotonicity and threshold differences.

Paired bootstrap intervals resample whole seed pairs, with 10,000 draws. The three-seed live intervals are exploratory, not population confidence about industrial deployment; no device-time pseudoreplication or global significance claim is used. Pilots with unfair service or earlier dispatch acknowledgement logic are excluded and explicitly labeled.

Source/config/evidence SHA-256 manifests, raw traces, dependency pins and regeneration scripts accompany this report.

10. Limitations and remaining external work

The reference core is an adapted method reproduction, not exact numerical replication. Its original captures and published attack figures were not reproduced. The later Linux extension executes official Docker services and bounded scenarios, with adaptations disclosed below. The native source remains separately fetched because no project-level license was verified. The first 108 live runs use application impairment; only the separate Linux sweep applies actual packet delay/loss. These initial local studies are followed by the separate AWS experiment in Section 13. The signal/workload is synthetic, clocks are co-located, runs are short, and desktop scheduling/storage alter service capacity. Timing/resource values vary across hosts.

Latest-value coalescing sacrifices intermediate live deliveries. A separate local audit preserves them but does not deliver them remotely. The audit currently has no retention policy. Receiver state and scheduling pointer are not persistent across processes; device epochs and end-to-end application acknowledgements are required for production restart guarantees. SQLite restart unit tests do not constitute physical power-loss tests. Overflow caps were not reached; differing live/model overflow choices are not evaluated.

Mentor approval is reported by the project team. Official Overleaf integration/access, group registration, human authorship/contribution review and defense remain external. Actual AWS edge/cloud deployment is now measured in Section 13; remote archive delivery and hardware/power experiments remain unmeasured. Physical hardware is optional unless required by the instructor, and no physical-validation claim is made. A full Overleaf source/image ZIP and tested deployment scripts are supplied. The course's final poster/presentation/submission workflow is separate from this phase report. No paper was submitted and no acceptance is implied.

12. Additional Linux/Docker and packet-impairment validation

After the initial report, Ubuntu/WSL2 was verified on this PC. Docker Desktop remained unavailable. Official Ubuntu packages were extracted into a private research runtime rather than installing a system Docker service; a separate Docker 29.1.3 daemon/socket/data directory was used. The official baseline Dockerfiles and source were built without modifying their Python method bodies. This installed current unpinned dependencies, including Paho 2.1.0 and aiocoap 0.4.17; these are not the paper's stated environment. Image IDs, source hashes and build/runtime logs are retained.

The first Docker attempt failed service-name resolution and generated no valid attack evidence. It is retained separately. The successful topology uses explicit private addresses/host mappings, an internal bridge, no exposed host ports, broker-first startup and service resource limits. The original approximately 66-second calibration completed without the native control-message repair. This does not reproduce the paper's 60-minute healthy capture.

The HTTP injection scenario produced three logged device sensor shutdowns. The MiTM proxy logged one benign-to-malicious command modification and one corresponding device temperature-sensor shutdown. A bounded single-worker five-second invocation of the original HTTP-flood function produced 1,033 logged device command receipts and 14 rate alerts in the observed post-attack window. The largest monitor-reported command rate was 237 application messages/s. These counters are not wire packets/s, and alerts alone do not establish classification accuracy. No confusion matrix or published Table 2 value is invented. Flood worker count/duration/resource limits differ from the unbounded official defaults.

The extension runs 36 real MQTT tests (three seeds x three QoS levels x FIFO/latest x healthy/delay-loss) in an isolated Docker network namespace with no external network. Four synthetic devices target 5Hz each for six seconds; the common application dispatch cap is 12/s. The packet-impairment condition applies 100ms delay with +/-20ms jitter and 2% configured loss to the namespace's loopback egress, affecting both MQTT connections. It is actual Linux tc/netem packet impairment, not a remote WAN or independently controlled link in each direction. Every run completes; the qdisc logs record 87 actual packet drops. Raw TCP/MQTT PCAP, per-run events, settings, resource measurements and SQLite audit snapshots are retained. All 4,248 generated samples are verified in audit snapshots; all 1,527 published messages are received. TCP recovery and finite-horizon edge suppression mean packet loss must not be interpreted as equal MQTT application-message loss.

Condition	Policy	QoS	Mean event age (s)	Receipt/generated (%)
delay_loss	fifo	0	2.950	23.89
delay_loss	fifo	1	3.271	12.92
delay_loss	fifo	2	3.440	8.41
delay_loss	latest	0	1.075	23.89
delay_loss	latest	1	1.371	12.19
delay_loss	latest	2	2.088	8.23
healthy	fifo	0	1.742	55.56
healthy	fifo	1	1.713	56.94

Condition	Policy	QoS	Mean event age (s)	Receipt/generated (%)
healthy	fifo	2	1.765	55.00
healthy	latest	0	0.279	56.94
healthy	latest	1	0.275	56.94
healthy	latest	2	0.282	55.56

Unlike the nearly continuous initial live sweep, packet delay and synchronous QoS waits create sparse, irregular in-loop samples. The largest difference between sampled and independently integrated mean age is 0.504 s. Consequently, the table uses event-integral device-time mean age as the primary metric, calculated from actual accepted receipts over seconds 1--6. This change prevents biased sample-cadence comparisons. Generated counts may be below the nominal 120 when a blocking publication reaches the horizon before the generator catches up; actual counts, not nominal counts, form each denominator.

The results support the FIFO-versus-latest freshness trade-off under executed packet impairment. QoS-specific throughput differences reflect this synchronous implementation, TCP behavior and the short local testbed; they are not universal MQTT-QoS rankings. Three seed repetitions are exploratory. Early build/attack work shared the host with the sweep, and the runner was capped at two CPUs/1GiB; resource values remain descriptive. AWS WAN evidence is provided in Section 13. Physical validation is not claimed; hardware is optional unless the instructor requires it. Six additional loopback integration checks confirm the prepared cloud publisher/receiver scripts work; those checks are explicitly not cloud results.

13. Executed AWS Internet validation

The temporary Ubuntu 24.04 t3.micro VM in Stockholm used Mosquitto 2.0.18 and Paho 2.1.0. 36 of 36 planned randomized runs completed. Four synthetic generators target 5Hz each for 20 seconds; FIFO/latest share a 12/s dispatch cap and nonblocking acknowledgement polling. MQTT traverses an SSH tunnel from the Windows edge PC to cloud localhost. The interruption condition closes and restores the owned tunnel at approximately seconds 8--12; it is not an ISP outage. No physical sensor or public-network attack is involved.

Every unique cloud receipt was checked against its source SQLite audit record. Generated samples total 14,256; unique cloud receipts total 3,709; publisher completions total 3,709. The sequence guard rejected 7 duplicate/stale callbacks. Publisher completion and cloud commit remain separate semantics.

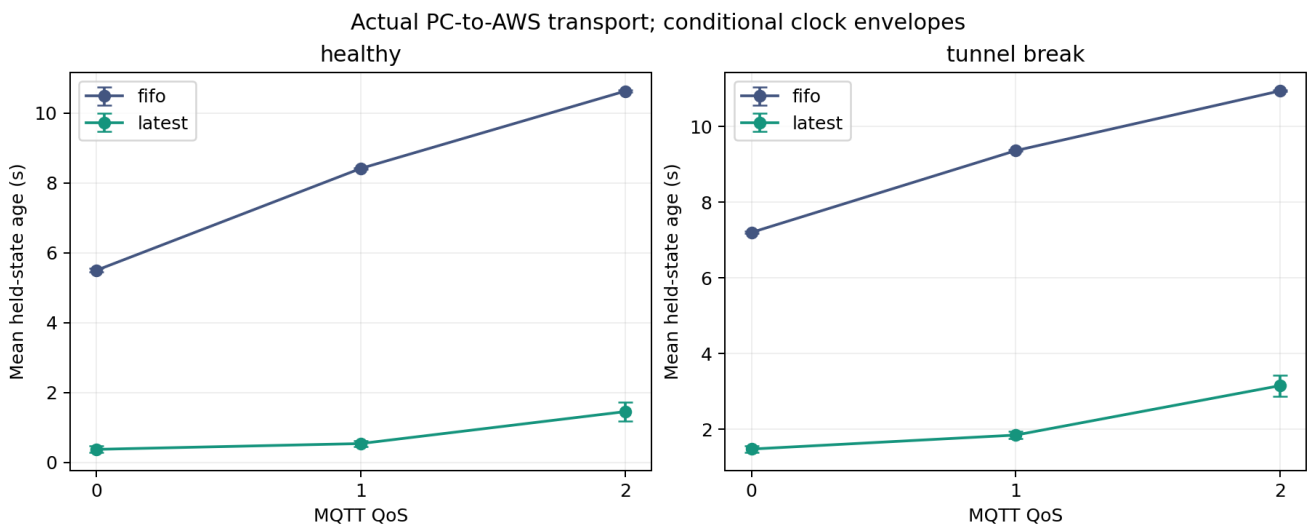
Scenario	QoS	Policy	Mean age (s)	Clock envelope (s)	Receipt ratio	Mean max queue
healthy	0	fifo	5.495	5.447--5.543	0.540	183
healthy	0	latest	0.370	0.280--0.460	0.540	4
healthy	1	fifo	8.410	8.384--8.437	0.288	283
healthy	1	latest	0.536	0.446--0.625	0.289	4
healthy	2	fifo	10.623	10.597--10.650	0.094	359
healthy	2	latest	1.449	1.177--1.725	0.095	4
tunnel_break	0	fifo	7.202	7.170--7.234	0.378	248
tunnel_break	0	latest	1.480	1.390--1.571	0.379	4
tunnel_break	1	fifo	9.363	9.347--9.379	0.193	320
tunnel_break	1	latest	1.850	1.760--1.940	0.197	4

Scenario	QoS	Policy	Mean age (s)	Clock envelope (s)	Receipt ratio	Mean max queue
tunnel_break	2	fifo	10.941	10.924--10.957	0.065	371
tunnel_break	2	latest	3.156	2.881--3.433	0.065	4

Age is integrated from accepted cloud receipts over seconds 3--20, with initial held timestamps set to run start. Before/after SSH time probes and receipt causality bound clock offset. A constant offset within each short run is assumed. Midpoint age and conditional envelopes are reported; these are not hardware-synchronized one-way measurements. Per-run CPU seconds and peak RSS are in the verified CSV. Raw databases, receiver logs, clock probes, actual tunnel timings and environment are retained. This small experiment complements the larger local sweep; MQTT-over-SSH, one region, one PC and short horizons limit generalization. See docs/AWS_WAN_VALIDATION.md for cleanup status and costs.

Process costs are averaged across conditions/QoS; RSS is the maximum observed for each policy. Receiver wall time includes startup, settle time and clock probing. These are edge/receiver process measurements and exclude broker, SSH tunnel and whole-machine consumption.

Policy	Edge wall (s)	Receiver wall (s)	Edge CPU (s)	Receiver CPU (s)	Edge RSS (MiB)	Receiver RSS (MiB)
fifo	20.10	34.67	0.457	0.073	25.85	24.36
latest	20.13	34.69	0.471	0.072	25.81	24.23



14. Conclusion

This research-phase package reproduces a feasible official MQTT communication core, exposes important paper/code differences, and implements a measurable current-state recovery extension. Under the executed constrained conditions, coalescing materially improves freshness over FIFO while leaving remote completeness as a separate concern. A fair unbuffered policy remains competitive, and established AoI/scheduling literature limits novelty claims. The defensible outcome is a transparent, reproducible evaluation and methodology foundation for instructor review and stronger external validation.

The additional AWS experiment confirms the same freshness-versus-history trade-off over actual Internet transport, with short-run clock uncertainty reported explicitly. All temporary cloud resources were removed after evidence transfer.

References

1. Rodrigues, Junior, Oliveira, Lima. A Data Rate Monitoring Approach for Cyberattack Detection in Digital Twin Communication. *Sensors* (2025). [DOI](#).
2. Cho, Noh. Design and Implementation of Digital Twin Factory Synchronized in Real-Time Using MQTT. *Machines* (2024). [DOI](#).
3. Crespo-Aguado, Lozano, Hernandez-Gobertti, Molner, Gomez-Barquero. Flexible Hyper-Distributed IoT–Edge–Cloud Platform for Real-Time Digital Twin Applications on 6G-Intended Testbeds for Logistics and Industry. *Future Internet* (2024). [DOI](#).
4. Li, Liang, Xu, Xu, Jia. Budget-Constrained Digital Twin Synchronization and Its Application on Fidelity-Aware Queries in Edge Computing. *IEEE Transactions on Mobile Computing* (2025). [DOI](#).
5. Dapkute, Siozinys, Jonaitis, Kaminickas, Siozinys. Digital Twin Data Management: Framework and Performance Metrics of Cloud-Based ETL System. *Machines* (2024). [DOI](#).
6. Neagu, Serban, Hangan, Sebestyen. Digital Twins at the Edge: A High-Availability Framework for Resilient Data Processing in IoT Sensor Networks. *Future Internet* (2026). [DOI](#).
7. Bellavista, Bicocchi, Fogli, Giannelli, Mamei, Picone. ODTE: A Metric for Digital Twin Entanglement. *IEEE Open Journal of the Communications Society* (2024). [DOI](#).
8. Loubany, Itani, Sharafeddine. From Age of Information to Age of Digital Twin: A Review on Synchronization Metrics for IoT Networks. *IEEE Access* (2025). [DOI](#).
9. Zhang, Wang, Li. Scheduling of Digital Twin Synchronization in Industrial Internet of Things: A Hybrid Inverse Reinforcement Learning Approach. *IEEE Internet of Things Journal* (2025). [DOI](#).
10. Guo, Fu, Zhang, Quek. Age-of-Information and Energy Optimization in Digital Twin Edge Networks. *GLOBECOM 2024 - 2024 IEEE Global Communications Conference* (2024). [DOI](#).
11. Bellavista, Giannelli, Mamei, Mendula, Picone. Application-Driven Network-Aware Digital Twin Management in Industrial Edge Environments. *IEEE Transactions on Industrial Informatics* (2021). [DOI](#).
12. Picone, Mamei, Zambonelli. WLDT: A general purpose library to build IoT digital twins. *SoftwareX* (2021). [DOI](#).
13. Infante, Martín, Robles, Rubio, Díaz, Perea, Montesinos, Poyato. Integrating FMI and ML/AI models on the open-source digital twin framework OpenTwins. *Software: Practice and Experience* (2024). [DOI](#).
14. Tan, Matta. The digital twin synchronization problem: Framework, formulations, and analysis. *IEEE Transactions* (2023). [DOI](#).
15. Herrnleben, Leidinger, Lesch, Prantl, Grohmann, Krupitzer, Kounev. ComBench: A Benchmarking Framework for Publish/Subscribe Communication Protocols Under Network Limitations. *Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering: Performance Evaluation Methodologies and Tools* (2021). [DOI](#).
16. Cakir, Al-Shareeda, Oktug, Özdem, Broadbent, Canberk. How to synchronize Digital Twins? A Communication Performance Analysis. *2023 IEEE 28th International Workshop on Computer Aided Modeling and Design of Communication Links and Networks (CAMAD)* (2023). [DOI](#).